

Báo cáo Lỗi và Vấn đề Hiệu năng (Bug & Performance Report)

- **Sinh viên:** Lê Trung Kiên - 23127075
- **Vai trò:** Thành viên 4, Admin Workflow
- **SUT:** EShop REST Backend (<http://localhost:3000>), Node.js và SQLite
- **Ngày ghi nhận:** 2026-09-03

1. Bug 1: Product CRUD thiếu kiểm soát truy cập (Access Control Bypass)

1.1. Tóm tắt

- **Mức độ:** High (Security Fault)
- **Phạm vi:** `POST /api/products`, `PUT /api/products/:id`, `DELETE /api/products/:id`
- **GitHub Issue:** #22 - [\[BUG-SEC-23127075\] Product CRUD endpoints lack authentication](#)
- **Screenshot:** [evidence/issues/github_issue_bug.png](#)

Ba endpoint thay đổi dữ liệu sản phẩm không gắn middleware `authenticateToken`, trong khi các endpoint quản trị tương tự như category CRUD và import products đều yêu cầu JWT Token. Người dùng vắng lai chưa xác thực vẫn có thể tạo, sửa hoặc xóa sản phẩm.

1.2. Các bước tái hiện

Tái hiện bằng câu lệnh `curl` không gửi header `Authorization`:

```
curl -X POST http://localhost:3000/api/products \
  -H 'Content-Type: application/json' \
  --data '{"name":"HW05 unauthenticated
probe","price":1,"description":"security
reproduction","imageUrl":"","category_id":1}'
```

Kết quả thực tế từ backend Node.js:

```
{"message":"Product created","id":8943}
```

Request trả HTTP 200 và tạo sản phẩm thành công. Dữ liệu probe sau đó được dọn bằng `DELETE /api/products/8943`, endpoint này cũng chấp nhận request không token và trả `{"message":"Product deleted"}`.

1.3. Kết quả mong đợi & Đề xuất sửa

Các endpoint ghi dữ liệu sản phẩm phải dùng `authenticateToken` và kiểm tra quyền admin (`req.user.role === 'admin'`). Request không có token hợp lệ phải bị từ chối với mã HTTP 401 hoặc

403.

2. Bug 2: SQL Injection tại API tìm kiếm sản phẩm (Input Validation Fault)

2.1. Tóm tắt

- **Mức độ:** High (Security Fault)
- **Phạm vi:** `GET /api/products?search=...`
- **GitHub Issue:** #23 - [BUG-SEC-23127075] SQL Injection Vulnerability in Product Search API

Mã nguồn backend tại `eshop-sut/backend/server.js` (dòng 144) sử dụng nối chuỗi SQL trực tiếp thay vì parameterized query: `const query = "SELECT * FROM products WHERE name LIKE '%" + searchQuery + "%'";` Điều này cho phép kẻ tấn công chèn cú pháp SQL hoặc làm crash câu truy vấn gây lỗi HTTP 500.

2.2. Các bước tái hiện

Gửi request tìm kiếm chứa ký tự nhảy đơn `'`:

```
curl "http://localhost:3000/api/products?search=phone'"
```

Kết quả thực tế backend sập câu SQL và trả về HTTP 500:

```
<h1>Database Error</h1><p>SQLITE_ERROR: near "'%': syntax error</p>
```

Hoặc khi gửi `curl "http://localhost:3000/api/products?search=' OR '1'='1'"`, truy vấn bị bypass và văng toàn bộ danh sách sản phẩm trong database.

2.3. Kết quả mong đợi & Đề xuất sửa

Sử dụng Parameterized Query chuẩn của SQLite: `db.all("SELECT * FROM products WHERE name LIKE ?", [%${searchQuery}%], ...)`

3. Bug 3: High Tail-Latency nghẽn SQLite dưới Spike Load (Performance Bottleneck)

3.1. Tóm tắt

- **Mức độ:** Medium (Performance Bottleneck)
- **Phạm vi:** Kịch bản Spike Testing (100 threads / 1s ramp-up)
- **GitHub Issue:** #24 - [BUG-PERF-23127075] Severe Tail-Latency Under Spike Load
- **Bảng chứng dữ liệu:** Log thô `src/results/spike/raw.jtl` và `src/validation-report.json`

Khi hệ thống chịu đợt Spike 100 Virtual Users trong 1 giây đồng thời thực hiện lệnh Create/Delete sản phẩm, mặc dù tỷ lệ lỗi HTTP bằng 0%, thời gian phản hồi ở đuôi (tail latency) suy giảm nghiêm trọng.

3.2. Số liệu chứng minh từ JTL

So sánh với các kịch bản khác trong bài làm:

Kịch bản	Threads / Ramp	Avg Response Time	p95 Response Time	p99 Response Time	Max Response Time
Load	10 VU / 10s	9.76 ms	17.0 ms	42.0 ms	61.0 ms
Stress	50 VU / 15s	7.62 ms	15.0 ms	20.0 ms	62.0 ms
Spike	100 VU / 1s	238.47 ms	476.0 ms	532.0 ms	569.0 ms

Độ trễ p95 tăng **gấp 31.7 lần** so với Stress test (từ 15ms lên 476ms).

3.3. Nguyên nhân & Đề xuất tối ưu

- **Nguyên nhân:** Hệ quản trị CSDL SQLite sử dụng cơ chế khóa ghi đơn (single-writer lock). Khi 100 thread ghi dữ liệu cùng lúc, các lệnh write phải xếp hàng (queueing delay), dẫn tới latency tích tụ.
- **Đề xuất tối ưu:** Bật chế độ WAL (Write-Ahead Logging) cho SQLite (`PRAGMA journal_mode=WAL;`) để cho phép đọc và ghi song song, hoặc chuyển sang PostgreSQL nếu cần phục vụ concurrency cao.